



**UNIVERSIDAD
DE GRANADA**

UNIVERSIDAD DE GRANADA

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación (ETSIIT)

Práctica 3

Autor:

Adrián Macías Caballero Paula
Genaro Soria

Asignatura:

Desarrollo de Aplicaciones en Red

14 de mayo de 2026

Índice

1. Introducción	3
2. Sistema de partida	3
3. Tabla de transformación de métodos a recursos REST	3
4. Diseño de la API REST	4
4.1. Criterio general de diseño	4
4.2. Recursos principales	5
5. Uso correcto de HTTP	5
5.1. Métodos HTTP utilizados	5
5.2. Códigos de estado	6
6. Diferencias respecto a objetos distribuidos	6
6.1. Modelo de comunicación	6
6.2. Serialización y legibilidad	6
6.3. Callbacks y notificaciones	7
7. Ventajas e inconvenientes de REST	7
7.1. Ventajas	7
7.2. Inconvenientes	8
8. Discusión técnica	8
8.1. Stateless	8
8.2. Diseño de URLs	8
8.3. Gestión de estado	9
9. Ejecución del servidor	9
9.1. Estructura del proyecto	9
9.2. Configuración del entorno	9
9.3. Instalación de dependencias	10
9.4. Arranque del servidor	10
9.5. Acceso a la documentación Swagger	10
10. Pruebas de funcionamiento	12
10.1. Comprobación básica del servidor	12
10.2. Consulta de ciudades	12
10.3. Consulta meteorológica	13
10.4. Creación de cliente	14
10.5. Creación de una suscripción	15
10.6. Consulta de suscripciones	15
10.7. Consulta de notificaciones	16
11. Pruebas con Wireshark	16
11.1. Filtro utilizado	16
11.2. Análisis de la primera captura	16
11.3. Análisis de la segunda captura	17
11.4. Análisis de la tercera captura	18
11.5. Interpretación global de las capturas	20
12. Problemas encontrados y soluciones adoptadas	20

1. Introducción

El objetivo de esta práctica ha sido transformar el sistema previamente desarrollado mediante sockets y, posteriormente, mediante objetos distribuidos en Java RMI, en un servicio web REST implementado en Python con el framework FastAPI. El sistema original, denominado MeteoApp, permitía consultar información meteorológica de distintas ciudades y gestionar suscripciones para recibir avisos cuando se producían cambios en determinadas variables meteorológicas.

La transición realizada no consiste simplemente en cambiar el lenguaje de programación de Java a Python. El cambio más importante está en el modelo de comunicación. En las versiones anteriores, el cliente se comunicaba con el servidor mediante un protocolo propio sobre sockets o mediante invocación remota de métodos usando RMI. En la nueva versión, la interacción se realiza mediante peticiones HTTP, utilizando JSON como formato de intercambio de datos.

Por tanto, el trabajo se centra en mantener la misma lógica funcional del sistema anterior, pero adaptándola a una arquitectura REST. Esto implica traducir las operaciones originales a recursos accesibles mediante URLs, elegir correctamente los métodos HTTP y devolver códigos de estado adecuados. También se ha tenido en cuenta que REST no debe utilizarse como si fuera una llamada remota disfrazada, por lo que se han evitado rutas basadas en acciones como `/getWeather` o `/subscribe`. En su lugar, se han diseñado recursos como `/cities`, `/weather/{city}` y `/clients/{client_id}/subscriptions`.

La implementación se ha completado con documentación automática mediante Swagger, pruebas de funcionamiento desde navegador y herramientas HTTP, y capturas de tráfico con Wireshark para comprobar que la comunicación se realiza realmente mediante HTTP y JSON.

2. Sistema de partida

El sistema original de MeteoApp estaba orientado a la monitorización meteorológica. En su primera versión, la comunicación se realizaba mediante sockets TCP y un protocolo propio basado en comandos. Posteriormente, en la versión con objetos distribuidos, las operaciones se encapsularon en una interfaz remota de Java RMI.

Desde el punto de vista funcional, el sistema ofrecía cuatro operaciones principales:

- Consultar la lista de ciudades disponibles.
- Consultar el estado meteorológico actual de una ciudad.
- Suscribirse a una ciudad para recibir cambios meteorológicos.
- Cancelar suscripciones existentes.

En la versión RMI, estas operaciones se expresaban como métodos remotos. Por ejemplo, el cliente podía invocar métodos como `listCities()`, `getWeather(city)`, `subscribe(city, callback)` o `unsubscribe(callback)`. Esta forma de trabajo resulta cómoda para el programador, porque la llamada remota se parece mucho a una llamada local. Sin embargo, internamente existe comunicación de red, serialización de objetos y dependencia de la tecnología Java RMI.

En la nueva versión REST, estas operaciones se han convertido en recursos HTTP. El cliente ya no invoca métodos remotos, sino que realiza peticiones a URLs concretas. La respuesta del servidor se devuelve en formato JSON y va acompañada de un código de estado HTTP que indica si la operación se ha realizado correctamente o si se ha producido algún error.

3. Tabla de transformación de métodos a recursos REST

La Tabla 1 muestra la transformación realizada desde las operaciones del sistema anterior hasta los nuevos endpoints REST.

Tabla 1: Transformación de operaciones del sistema anterior a recursos REST.

Operación anterior	ante-	Semántica	Endpoint REST	Método HTTP	Código es-
LIST listCities()	/	Obtener el catálogo de ciudades disponibles en el sistema.	/cities	GET	200 OK
GET getWeather(city)	/	Consultar el estado meteorológico actual de una ciudad concreta.	/weather/{city}	GET	200 OK o 404 Not Found
GET sin ciudad explícita		Consultar el tiempo de una ciudad indicada mediante parámetro.	/weather?city=Madrid	GET	200 OK
SUB subscribe(city, callback)	/	Crear una suscripción meteorológica asociada a un cliente.	/clients/{client_id}/subscriptions	POST	201 Created
SUB duplicado		Intentar suscribirse dos veces a la misma ciudad.	/clients/{client_id}/subscriptions	POST	400 Bad Request
UNSUB total unsubscribe(callback)	/	Cancelar todas las suscripciones asociadas a un cliente.	/clients/{client_id}/subscriptions	DELETE	200 OK
UNSUB con ciudad		Cancelar únicamente la suscripción de una ciudad concreta.	/clients/{client_id}/subscriptions/{city}	DELETE	200 OK o 404 Not Found
NOTIF onWeatherChange(data)	/	Consultar cambios meteorológicos pendientes para un cliente.	/clients/{client_id}/notifications	GET	200 OK

La transformación se ha realizado intentando respetar la semántica original de las operaciones, pero adaptándola al estilo REST. Por ejemplo, la operación de listar ciudades no se modela como /listCities, sino como una consulta sobre el recurso /cities. Del mismo modo, la suscripción no se implementa como /subscribe, sino como la creación de un recurso dentro de la colección de suscripciones de un cliente.

4. Diseño de la API REST

4.1. Criterio general de diseño

El criterio principal seguido en el diseño de la API ha sido representar recursos y no acciones. En REST, las URLs deben identificar entidades del sistema, mientras que la acción que se realiza sobre ellas debe venir determinada por el método HTTP utilizado.

Por este motivo, se han evitado rutas como:

```
/getWeather
/listCities
/subscribe
/unsubscribe
/sendNotification
```

Estas rutas son propias de un diseño RPC, ya que el nombre de la operación aparece directamente en la URL. En su lugar, se han utilizado rutas basadas en recursos:

```
/cities
/weather/{city}
/clients
/clients/{client_id}/subscriptions
/clients/{client_id}/notifications
```

De esta manera, la API queda más alineada con los principios REST. Por ejemplo, una petición GET sobre `/cities` obtiene ciudades, mientras que una petición POST sobre `/clients/{client_id}/subscriptions` crea una nueva suscripción para ese cliente.

4.2. Recursos principales

Los recursos principales definidos en la API son los siguientes:

- **Ciudades:** representan el conjunto de ciudades disponibles para consulta meteorológica.
- **Tiempo meteorológico:** representa el estado actual de una ciudad concreta.
- **Clientes:** representan a los consumidores de la API que desean mantener suscripciones.
- **Suscripciones:** representan la relación entre un cliente y una ciudad monitorizada.
- **Notificaciones:** representan los cambios meteorológicos pendientes de ser consultados por un cliente.

La introducción del recurso `/clients` es especialmente importante. En RMI, el cliente podía identificarse mediante una referencia remota de callback. En REST, esa referencia no existe, por lo que se utiliza un identificador explícito, denominado `client_id`. Este identificador se obtiene al crear un cliente mediante POST `/clients` y se usa posteriormente en las operaciones de suscripción y consulta de notificaciones.

5. Uso correcto de HTTP

5.1. Métodos HTTP utilizados

En la implementación se han utilizado los métodos HTTP de acuerdo con el tipo de operación realizada:

- GET: se utiliza para consultar información sin modificar el estado del servidor.
- POST: se utiliza para crear nuevos recursos, como clientes o suscripciones.
- DELETE: se utiliza para eliminar recursos, como suscripciones activas.

Por ejemplo, consultar la lista de ciudades se realiza mediante:

```
GET /cities
```

Crear un cliente se realiza mediante:

```
POST /clients
```

Crear una suscripción se realiza mediante:

```
POST /clients/{client_id}/subscriptions
```

Y cancelar una suscripción concreta se realiza mediante:

```
DELETE /clients/{client_id}/subscriptions/{city}
```

5.2. Códigos de estado

También se han utilizado códigos de estado HTTP para indicar el resultado de cada operación. Esto es una diferencia importante respecto al protocolo propio anterior, donde era habitual incluir el resultado únicamente dentro del mensaje de respuesta.

Los códigos principales utilizados son:

- 200 OK: la petición se ha procesado correctamente.
- 201 Created: se ha creado un nuevo recurso, como un cliente o una suscripción.
- 400 Bad Request: la petición contiene datos incorrectos o se intenta realizar una operación no válida, como una suscripción duplicada.
- 404 Not Found: el recurso solicitado no existe.
- 405 Method Not Allowed: se ha usado un método HTTP no permitido para un endpoint concreto.
- 500 Internal Server Error: se ha producido un error interno, por ejemplo si no está configurada la clave de la API meteorológica.

El uso de estos códigos hace que la API sea más clara para clientes externos, ya que no es necesario interpretar únicamente el contenido JSON para saber si una operación ha fallado.

6. Diferencias respecto a objetos distribuidos

6.1. Modelo de comunicación

En la versión con objetos distribuidos, el cliente invocaba métodos remotos como si fueran métodos Java ordinarios. La comunicación quedaba parcialmente oculta por RMI: el cliente obtenía una referencia remota, invocaba un método y recibía una respuesta. Internamente, RMI se encargaba de transportar la invocación, serializar los objetos necesarios y devolver el resultado.

En REST, la comunicación es explícita. El cliente construye una petición HTTP, la envía a una URL determinada y recibe una respuesta HTTP. Esta respuesta incluye un código de estado, cabeceras y, normalmente, un cuerpo en formato JSON.

La diferencia conceptual es importante. En RMI se piensa en términos de métodos remotos; en REST se piensa en términos de recursos. Por ejemplo, en RMI se invoca:

```
getWeather("Granada")
```

Mientras que en REST se realiza:

```
GET /weather/Granada
```

Ambas operaciones tienen la misma finalidad funcional, pero pertenecen a modelos de comunicación distintos.

6.2. Serialización y legibilidad

En RMI, los datos viajan serializados mediante mecanismos propios de Java. Esto hace que el tráfico sea menos legible cuando se inspecciona con herramientas como Wireshark. En cambio, en la versión REST, la información viaja sobre HTTP y se codifica en JSON. Esto permite ver con mayor claridad la petición, la respuesta, el endpoint utilizado y los datos transmitidos.

Esta diferencia se aprecia claramente en las capturas realizadas con Wireshark. En la versión REST aparecen líneas como:

```
GET /weather/Granada HTTP/1.1
HTTP/1.1 200 OK
POST /clients HTTP/1.1
HTTP/1.1 201 Created
Content-Type: application/json
```

Esto facilita la depuración y permite comprobar directamente si la API está utilizando correctamente los métodos HTTP y los códigos de estado.

6.3. Callbacks y notificaciones

La diferencia más delicada se encuentra en el mecanismo de notificaciones. En RMI, el servidor podía invocar un método remoto del cliente, por ejemplo `onWeatherChange(data)`, cuando se detectaba un cambio meteorológico. Esto encaja bien con el modelo de objetos distribuidos, porque el cliente también puede exponer un objeto remoto.

En REST, el servidor no invoca directamente métodos del cliente. El modelo habitual es petición-respuesta, iniciado por el cliente. Por esta razón, el callback remoto se ha transformado en un recurso consultable:

```
GET /clients/{client_id}/notifications
```

El servidor mantiene internamente una cola de notificaciones pendientes para cada cliente. Cuando el cliente quiere comprobar si hay cambios, consulta ese recurso. Esta solución mantiene la lógica funcional del sistema, aunque cambia la forma de interacción: se pasa de un modelo de notificación directa a un modelo de consulta periódica o *polling*.

7. Ventajas e inconvenientes de REST

7.1. Ventajas

Una de las principales ventajas de REST es la interoperabilidad. Al utilizar HTTP y JSON, el cliente y el servidor no tienen que estar implementados en el mismo lenguaje. Un servidor desarrollado en Python con FastAPI puede ser consumido por un cliente Java, JavaScript, Python, una aplicación móvil o una herramienta como Bruno o `curl`.

Otra ventaja importante es la facilidad de prueba. En esta práctica se ha podido probar la API desde el navegador mediante Swagger, desde línea de comandos con `curl` y desde herramientas específicas para APIs REST. Esto contrasta con RMI, donde el cliente debe conocer la interfaz remota y trabajar dentro del ecosistema Java.

También resulta más sencilla la depuración del tráfico. Al estar basado en HTTP, las peticiones y respuestas se pueden capturar e interpretar con Wireshark de forma directa. En las capturas se observa el método utilizado, la ruta solicitada, el código de estado devuelto y el tipo de contenido, como `application/json`.

Otra ventaja es la documentación automática. FastAPI genera una especificación OpenAPI a partir de los endpoints definidos y de los modelos de datos utilizados. Gracias a ello, la API queda documentada en una interfaz Swagger accesible desde:

```
http://127.0.0.1:8000/docs
```

También se puede acceder a la documentación alternativa ReDoc mediante:

```
http://127.0.0.1:8000/redoc
```


7.2. Inconvenientes

El principal inconveniente de REST en este sistema es que no reproduce de forma natural el mecanismo de callback de RMI. En la versión con objetos distribuidos, el servidor podía avisar directamente al cliente. En REST, si se desea mantener un diseño sencillo y basado en HTTP, el cliente debe consultar periódicamente si tiene notificaciones pendientes.

Esto puede introducir cierta latencia. Si el cliente consulta cada 60 segundos, un cambio meteorológico puede tardar hasta 60 segundos en ser recibido. Además, se generan más peticiones HTTP, incluso cuando no hay cambios disponibles.

Otro inconveniente es la gestión del estado. Aunque HTTP es stateless, el sistema necesita mantener información de clientes, suscripciones y notificaciones pendientes. Esto obliga a diseñar explícitamente cómo se identifica cada cliente y cómo se conserva su estado en el servidor.

A pesar de estos inconvenientes, REST resulta adecuado para esta práctica porque permite transformar el sistema original en una API clara, documentada, portable y fácilmente verificable mediante herramientas estándar.

8. Discusión técnica

8.1. Stateless

HTTP se considera un protocolo stateless porque cada petición debe contener la información necesaria para que el servidor pueda procesarla. El servidor no debe depender de una conexión persistente para interpretar la petición actual.

En el sistema original basado en sockets, podía existir una relación más directa entre conexión y cliente. En RMI, la identidad del cliente podía estar asociada a la referencia remota utilizada como callback. Sin embargo, en REST no se debe asumir que una conexión TCP concreta representa a un cliente durante toda la ejecución.

Para resolver esto, se ha introducido el identificador `client_id`. El flujo es el siguiente:

1. El cliente realiza una petición `POST /clients`.
2. El servidor crea un identificador único.
3. El cliente utiliza ese identificador en las siguientes peticiones.
4. Las suscripciones y notificaciones quedan asociadas a ese `client_id`.

De esta forma, aunque HTTP sea stateless, la aplicación puede mantener estado de negocio de manera controlada.

8.2. Diseño de URLs

El diseño de URLs ha sido uno de los aspectos más importantes de la práctica. En REST, una URL debe representar un recurso, no una acción. Por ello, se ha evitado utilizar nombres de métodos en las rutas.

Por ejemplo, una solución incorrecta habría sido:

```
POST /subscribe
POST /unsubscribe
GET /getWeather
GET /listCities
```

Aunque estas rutas funcionarían técnicamente, no representarían un diseño REST adecuado. La solución adoptada utiliza rutas más expresivas:

```
GET    /cities
GET    /weather/{city}
POST   /clients
POST   /clients/{client_id}/subscriptions
GET    /clients/{client_id}/subscriptions
GET    /clients/{client_id}/notifications
DELETE /clients/{client_id}/subscriptions/{city}
```

En este diseño, la ruta identifica el recurso y el método HTTP indica la operación.

8.3. Gestión de estado

El servidor mantiene varios elementos de estado interno:

- Clientes registrados.
- Suscripciones activas de cada cliente.
- Último valor meteorológico conocido para cada suscripción.
- Cola de notificaciones pendientes.

Esta información permite conservar la lógica de la práctica anterior. Por ejemplo, si un cliente ya está suscrito a una ciudad, el servidor puede detectar la operación duplicada y devolver un `400 Bad Request`. Si el cliente consulta sus notificaciones, el servidor puede devolver los cambios pendientes en formato JSON.

La gestión de estado se ha trasladado desde la conexión o el callback remoto a estructuras internas asociadas al `client_id`. Este cambio es necesario para adaptar el sistema al modelo REST.

9. Ejecución del servidor

9.1. Estructura del proyecto

La versión REST se ha organizado en un nuevo directorio del repositorio. La estructura básica del proyecto es la siguiente:

```
meteo_rest_fastapi/
|-- app/
|   |-- main.py
|-- requirements.txt
|-- .env.example
|-- README.md
|-- curl_examples.md
|-- docs/
|   |-- memoria_rest.md
```

El fichero principal de la aplicación es `app/main.py`. En él se define la instancia de FastAPI, los modelos de datos, las rutas de la API y la lógica necesaria para consultar información meteorológica y gestionar clientes, suscripciones y notificaciones.

9.2. Configuración del entorno

Para ejecutar el proyecto es necesario instalar las dependencias indicadas en el fichero `requirements.txt`. Antes de arrancar el servidor, se debe crear un fichero `.env` en la raíz del proyecto. Este fichero contiene la clave de la API meteorológica y otros parámetros de configuración.

Un ejemplo de fichero `.env` sería:

```
OPENWEATHER_API_KEY=tu_clave_real_de_openweather
UPDATE_INTERVAL_SECONDS=60
```

Es importante que el fichero se llame exactamente `.env` y no `.env.txt`. Además, no debe subirse al repositorio si contiene claves privadas. En su lugar, puede subirse un fichero `.env.example` con valores de ejemplo.

9.3. Instalación de dependencias

Desde la carpeta `meteo_rest_fastapi`, se crea y activa un entorno virtual. En Windows PowerShell, los comandos son:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Una vez activado el entorno virtual, la terminal suele mostrar el prefijo `(.venv)`, indicando que los paquetes se instalarán dentro de ese entorno y no globalmente en el sistema.

9.4. Arranque del servidor

El servidor se ejecuta con `uvicorn`. Para que sea accesible desde otra máquina o desde una máquina virtual, se utiliza la opción `-host 0.0.0.0`:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

El parámetro `app.main:app` indica que se debe cargar el objeto `app` definido en el fichero `app/main.py`. El puerto utilizado es el 8000. La opción `-reload` permite reiniciar automáticamente el servidor cuando se modifica el código, lo cual resulta útil durante el desarrollo.

Si el servidor arranca correctamente, se muestra un mensaje similar al siguiente:

```
Uvicorn running on http://0.0.0.0:8000
Application startup complete.
```



```
[notice] A new release of pip is available: 24.2 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Usuario\Desktop\UNI\Cuarto de Carrera Teleco\Segundo Cuatri\meteo_rest_fastapi\meteo_rest_fastapi> uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
INFO: Will watch for changes in these directories: ['C:\Users\Usuario\Desktop\UNI\Cuarto de Carrera Teleco\Segundo Cuatri\meteo_rest_fastapi\meteo_rest_fastapi']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [7724] using WatchFiles
INFO: Started server process [4268]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figura 1: Inicio del servidor.

9.5. Acceso a la documentación Swagger

Una vez arrancado el servidor, se puede acceder a Swagger desde el navegador:

```
http://127.0.0.1:8000/docs
```

Desde Swagger se pueden probar directamente los endpoints de la API. También está disponible la documentación ReDoc:

```
http://127.0.0.1:8000/redoc
```

Y el documento OpenAPI en formato JSON:

```
http://127.0.0.1:8000/openapi.json
```

MeteoApp REST API 1.0.0 OAS 3.1

[OpenAPI JSON](#)

Transformación del sistema de monitorización meteorológica basado en sockets/RMI a una API REST con HTTP y JSON. La API mantiene las operaciones LIST, GET, SUB y UNSUB como recursos REST.

Uso académico

Sistema ^	
GET	/health Health
Ciudades ^	
GET	/cities List Cities
Meteorología ^	
GET	/weather Get Default Weather
GET	/weather/{city} Get Weather
Clientes ^	
POST	/clients Create Client
DELETE	/clients/{client_id} Delete Client

Figura 2: Documentación Swagger.

Suscripciones ^	
POST	/clients/{client_id}/subscriptions Create Subscription
GET	/clients/{client_id}/subscriptions List Subscriptions
DELETE	/clients/{client_id}/subscriptions Delete All Subscriptions
DELETE	/clients/{client_id}/subscriptions/{city} Delete Subscription
Notificaciones ^	
GET	/clients/{client_id}/notifications Get Notifications

Figura 3: Documentación Swagger.

Schemas ^	
CityListResponse	Expand all object
ClientCreateResponse	Expand all object
HTTPValidationError	Expand all object
MessageResponse	Expand all object
Notification	Expand all object
NotificationListResponse	Expand all object
SubscriptionCreate	Expand all object
SubscriptionInfo	Expand all object
SubscriptionListResponse	Expand all object
SubscriptionResponse	Expand all object
ValidationError	Expand all object
WeatherResponse	Expand all object
WeatherValues	Expand all object

Figura 4: Documentación Swagger.

10. Pruebas de funcionamiento

10.1. Comprobación básica del servidor

La primera prueba consiste en comprobar que el servidor está activo mediante el endpoint de salud:

```
GET /health
```

La respuesta esperada es un JSON indicando que el servidor está funcionando correctamente:

```
{
  "status": 200,
  "msg": "Servidor REST activo"
}
```

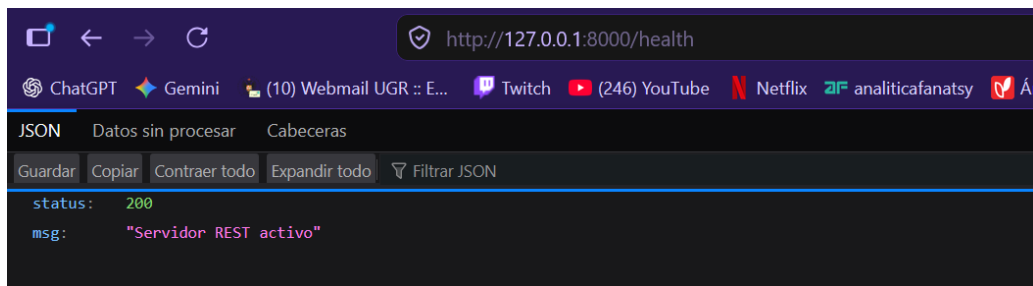


Figura 5: Comprobación punto de salud.

10.2. Consulta de ciudades

Para obtener las ciudades disponibles, se utiliza:

```
GET /cities
```

Esta operación no modifica el estado del servidor, por lo que se implementa mediante **GET**. La respuesta correcta debe devolver un código 200 OK y un cuerpo JSON con la lista de ciudades.

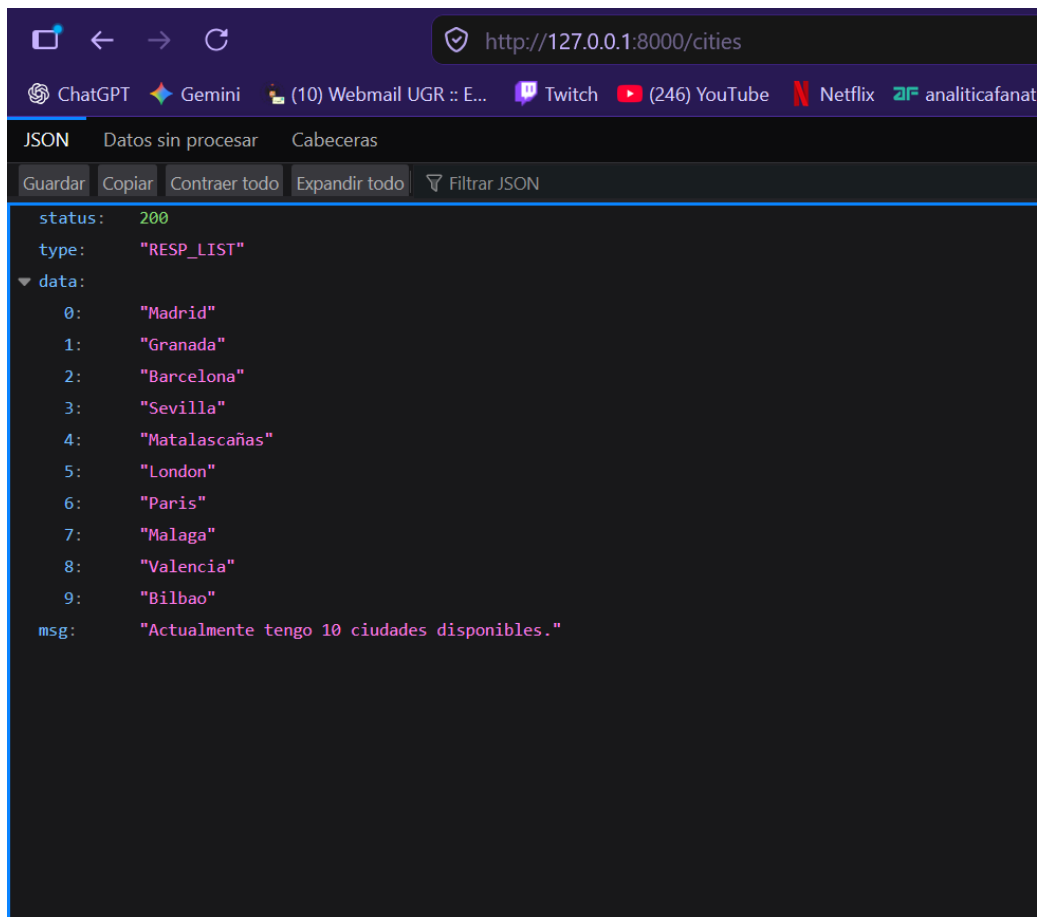


Figura 6: Consulta de ciudades.

10.3. Consulta meteorológica

Para consultar el tiempo de una ciudad concreta, se utiliza:

```
GET /weather/Granada
```

Si la ciudad existe y la clave de OpenWeather está correctamente configurada, el servidor devuelve un código 200 OK y un JSON con la información meteorológica. Si la variable de entorno `OPENWEATHER_API_KEY` no está definida, el servidor devuelve un error indicando que falta la configuración necesaria.

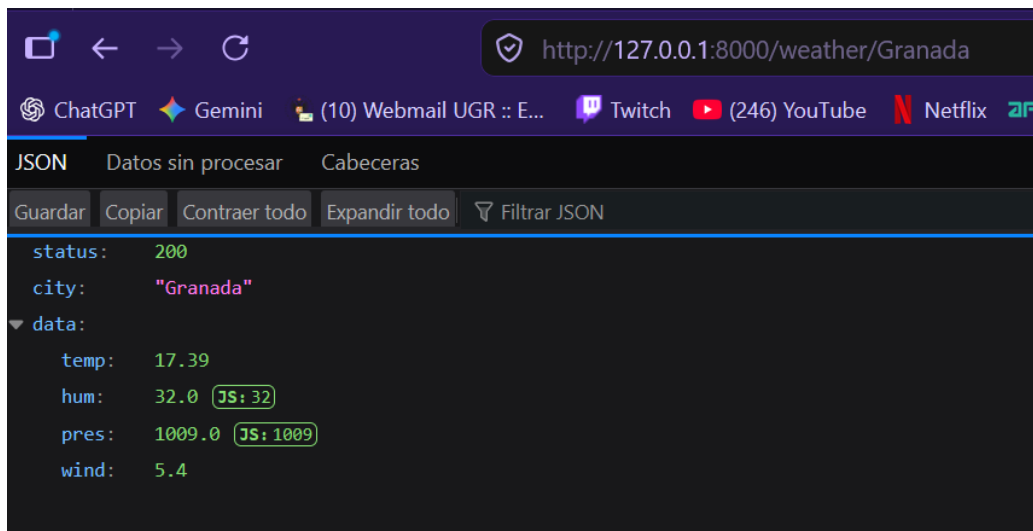


Figura 7: Consulta meteorológica.

10.4. Creación de cliente

Para crear un cliente REST se ejecuta:

```
POST /clients
```

La respuesta esperada es un código 201 Created, junto con un identificador de cliente:

```

{
  "status": 201,
  "client_id": "0dcc75-f3d-4db-a31c-90a2c771cb7d",
  "msg": "Cliente REST creado"
}

```

Este identificador debe copiarse para realizar las operaciones posteriores de suscripción.

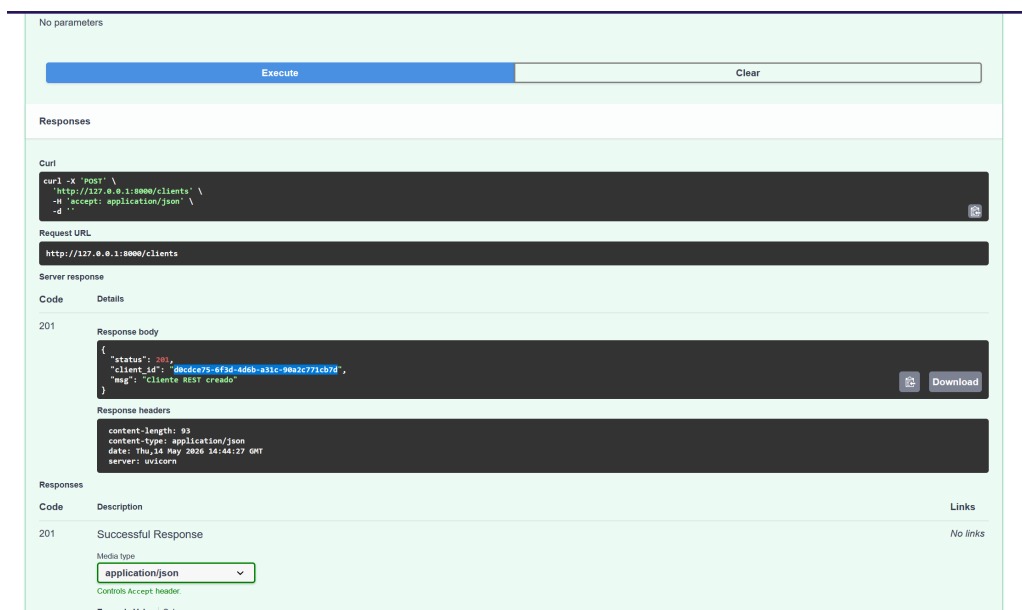


Figura 8: Creación del cliente.

client_id * required
string
(path)

d0cde75-6f3d-4d6b-a31c-90a2c771cb7d

Request body required application/json

Edit Value Schema

```
{
  "city": "Granada",
  "variables": ["temp", "hum", "pres", "wind"]
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/clients/d0cde75-6f3d-4d6b-a31c-90a2c771cb7d/subscriptions' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "city": "Granada",
    "variables": ["temp", "hum", "pres", "wind"]
  }'
```

Request URL

```
http://127.0.0.1:8000/clients/d0cde75-6f3d-4d6b-a31c-90a2c771cb7d/subscriptions
```

Figura 9: Creación del cliente.

10.5. Creación de una suscripción

Una vez creado el cliente, se puede crear una suscripción mediante:

```
POST /clients/{client_id}/subscriptions
```

El cuerpo de la petición se envía en JSON. Un ejemplo válido sería:

```
{
  "city": "Granada",
  "variables": ["temp", "hum", "pres", "wind"]
}
```

Si la operación se realiza correctamente, el servidor devuelve **201 Created**. Si se intenta repetir la misma suscripción para el mismo cliente y ciudad, el servidor devuelve **400 Bad Request**, ya que se trata de una operación duplicada.

Code Details

400
Undocumented

Error: Bad Request

Response body

```
{
  "status": 400,
  "msg": "Ya está suscrito a las alertas de Granada. 🚫"
}
```

Response headers

```
content-length: 71
content-type: application/json
date: Thu, 14 May 2020 14:45:58 GMT
server: uvicorn
```

Download

Figura 10: Comprobación suscripción.

10.6. Consulta de suscripciones

Para comprobar las suscripciones activas de un cliente se utiliza:

```
GET /clients/{client_id}/subscriptions
```

La respuesta correcta devuelve **200 OK** y una lista de las ciudades a las que el cliente está suscrito.

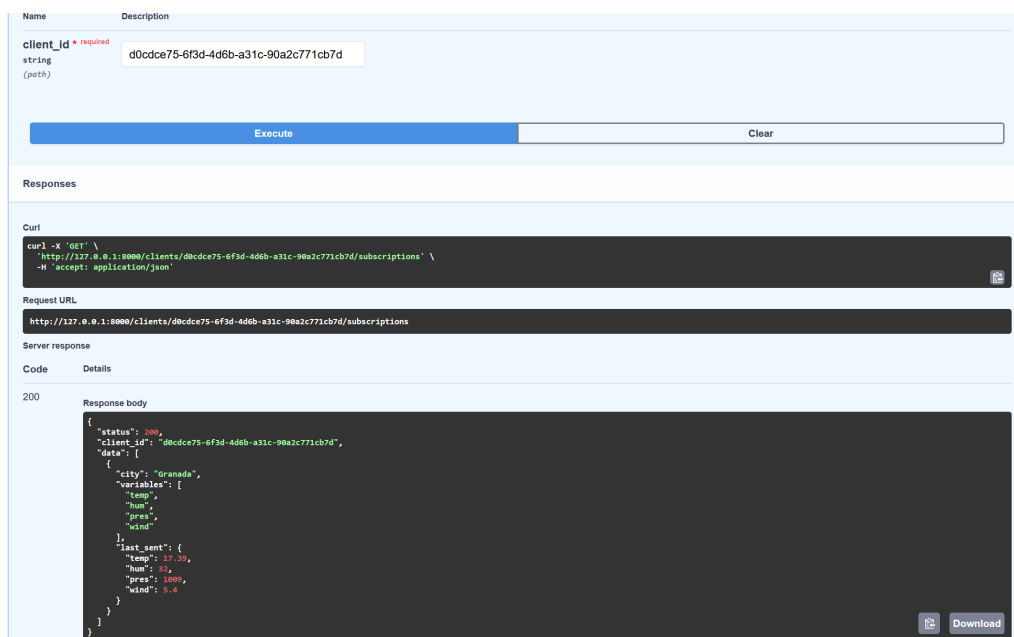


Figura 11: Consulta subscripciones

10.7. Consulta de notificaciones

Las notificaciones se consultan mediante:

```
GET /clients/{client_id}/notifications
```

Esta ruta sustituye al callback remoto de RMI. En lugar de que el servidor invoque un método del cliente, el cliente consulta periódicamente este recurso para comprobar si existen cambios pendientes.

11. Pruebas con Wireshark

11.1. Filtro utilizado

Para capturar el tráfico de la API REST se ha utilizado Wireshark aplicando el filtro:

```
tcp.port == 8000
```

Este filtro permite mostrar únicamente los paquetes TCP relacionados con el servidor FastAPI, que se está ejecutando en el puerto 8000. Como las pruebas se han realizado desde la misma máquina, el origen y destino observados son 127.0.0.1. Esto indica que el tráfico se está produciendo a través de la interfaz de loopback.

También se podría utilizar un filtro más específico si se quisiera mostrar solo tráfico HTTP:

```
http && tcp.port == 8000
```

Sin embargo, el filtro `tcp.port == 8000` es más seguro, ya que muestra toda la comunicación aunque Wireshark no clasifique automáticamente todos los paquetes como HTTP.

11.2. Análisis de la primera captura

En la Figura 12 se observa tráfico HTTP generado durante la consulta meteorológica y durante el acceso a la documentación de la API.

No.	Time	Source	Destination	Protocol	Length	Info
493	10.588769	127.0.0.1	127.0.0.1	TCP	56	57927 → 8000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
494	10.588848	127.0.0.1	127.0.0.1	TCP	56	8000 → 57927 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
495	10.588885	127.0.0.1	127.0.0.1	TCP	44	57927 → 8000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
500	10.593803	127.0.0.1	127.0.0.1	HTTP	528	GET /weather/Granada HTTP/1.1
501	10.593834	127.0.0.1	127.0.0.1	TCP	44	8000 → 57927 [ACK] Seq=1 Ack=485 Win=65824 Len=0
513	11.218153	127.0.0.1	127.0.0.1	TCP	169	8000 → 57927 [PSH, ACK] Seq=1 Ack=485 Win=65824 Len=125 [TCP PDU reassembled in 515]
514	11.218195	127.0.0.1	127.0.0.1	TCP	44	57927 → 8000 [ACK] Seq=485 Ack=126 Win=65280 Len=0
515	11.218225	127.0.0.1	127.0.0.1	HTTP/1.1	133	200 OK, JSON (application/json)
516	11.218238	127.0.0.1	127.0.0.1	TCP	44	57927 → 8000 [ACK] Seq=485 Ack=215 Win=65280 Len=0
521	11.238988	127.0.0.1	127.0.0.1	TCP	56	57931 → 8000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
522	11.239045	127.0.0.1	127.0.0.1	TCP	56	8000 → 57931 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
523	11.239091	127.0.0.1	127.0.0.1	TCP	44	57931 → 8000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
526	11.239266	127.0.0.1	127.0.0.1	HTTP	559	GET /favicon.ico HTTP/1.1
527	11.239281	127.0.0.1	127.0.0.1	TCP	44	8000 → 57931 [ACK] Seq=1 Ack=516 Win=64768 Len=0
528	11.240559	127.0.0.1	127.0.0.1	TCP	176	8000 → 57931 [PSH, ACK] Seq=1 Ack=516 Win=64768 Len=132 [TCP PDU reassembled in 530]
529	11.240605	127.0.0.1	127.0.0.1	TCP	44	57931 → 8000 [ACK] Seq=516 Ack=133 Win=65280 Len=0
530	11.240657	127.0.0.1	127.0.0.1	HTTP/1.1	66	404 Not Found, JSON (application/json)
531	11.240677	127.0.0.1	127.0.0.1	TCP	44	57931 → 8000 [ACK] Seq=516 Ack=155 Win=65280 Len=0
627	16.204542	127.0.0.1	127.0.0.1	TCP	44	8000 → 57927 [FIN, ACK] Seq=215 Ack=485 Win=65824 Len=0
628	16.204582	127.0.0.1	127.0.0.1	TCP	44	57927 → 8000 [ACK] Seq=485 Ack=216 Win=65280 Len=0
629	16.204757	127.0.0.1	127.0.0.1	TCP	44	57927 → 8000 [FIN, ACK] Seq=485 Ack=216 Win=65280 Len=0
630	16.204804	127.0.0.1	127.0.0.1	TCP	44	8000 → 57927 [ACK] Seq=216 Ack=486 Win=65824 Len=0
638	16.250662	127.0.0.1	127.0.0.1	TCP	44	8000 → 57931 [FIN, ACK] Seq=155 Ack=516 Win=64768 Len=0
639	16.250691	127.0.0.1	127.0.0.1	TCP	44	57931 → 8000 [ACK] Seq=516 Ack=156 Win=65280 Len=0
640	16.250894	127.0.0.1	127.0.0.1	TCP	44	57931 → 8000 [FIN, ACK] Seq=156 Ack=156 Win=65280 Len=0
641	16.250917	127.0.0.1	127.0.0.1	TCP	44	8000 → 57931 [ACK] Seq=156 Ack=517 Win=64768 Len=0
644	16.291825	127.0.0.1	127.0.0.1	TCP	56	57952 → 8000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
645	16.291876	127.0.0.1	127.0.0.1	TCP	56	8000 → 57952 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
646	16.291901	127.0.0.1	127.0.0.1	TCP	44	57952 → 8000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
664	16.701463	127.0.0.1	127.0.0.1	HTTP	519	GET /cities HTTP/1.1
665	16.701526	127.0.0.1	127.0.0.1	TCP	44	8000 → 57952 [ACK] Seq=1 Ack=476 Win=65824 Len=0
668	16.703268	127.0.0.1	127.0.0.1	TCP	176	8000 → 57952 [PSH, ACK] Seq=1 Ack=476 Win=65824 Len=126 [TCP PDU reassembled in 670]
669	16.703289	127.0.0.1	127.0.0.1	TCP	44	57952 → 8000 [ACK] Seq=476 Ack=127 Win=65280 Len=0
670	16.703323	127.0.0.1	127.0.0.1	HTTP/1.1	248	200 OK, JSON (application/json)
671	16.703337	127.0.0.1	127.0.0.1	TCP	44	57952 → 8000 [ACK] Seq=476 Ack=323 Win=65824 Len=0
676	16.740286	127.0.0.1	127.0.0.1	TCP	56	57955 → 8000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
678	16.740395	127.0.0.1	127.0.0.1	TCP	56	8000 → 57955 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
680	16.740453	127.0.0.1	127.0.0.1	TCP	44	57955 → 8000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
681	16.740702	127.0.0.1	127.0.0.1	HTTP	558	GET /favicon.ico HTTP/1.1
682	16.740727	127.0.0.1	127.0.0.1	TCP	44	8000 → 57955 [ACK] Seq=1 Ack=507 Win=65824 Len=0
687	16.742268	127.0.0.1	127.0.0.1	TCP	176	8000 → 57955 [PSH, ACK] Seq=1 Ack=507 Win=65824 Len=132 [TCP PDU reassembled in 689]
688	16.742312	127.0.0.1	127.0.0.1	TCP	44	57955 → 8000 [ACK] Seq=507 Ack=133 Win=65280 Len=0
689	16.742394	127.0.0.1	127.0.0.1	HTTP/1.1	66	404 Not Found, JSON (application/json)
690	16.742419	127.0.0.1	127.0.0.1	TCP	44	57955 → 8000 [ACK] Seq=507 Ack=155 Win=65280 Len=0
827	21.696456	127.0.0.1	127.0.0.1	TCP	44	8000 → 57955 [ACK] Seq=324 Ack=477 Win=65824 Len=0
828	21.696497	127.0.0.1	127.0.0.1	TCP	44	57952 → 8000 [ACK] Seq=476 Ack=324 Win=65824 Len=0
829	21.696585	127.0.0.1	127.0.0.1	TCP	44	57952 → 8000 [FIN, ACK] Seq=476 Ack=324 Win=65824 Len=0
830	21.696628	127.0.0.1	127.0.0.1	TCP	44	8000 → 57952 [ACK] Seq=324 Ack=477 Win=65824 Len=0
833	21.759855	127.0.0.1	127.0.0.1	TCP	44	8000 → 57955 [FIN, ACK] Seq=155 Ack=507 Win=65824 Len=0
832	21.758882	127.0.0.1	127.0.0.1	TCP	44	57955 → 8000 [ACK] Seq=507 Ack=156 Win=65280 Len=0
833	21.759835	127.0.0.1	127.0.0.1	TCP	44	57955 → 8000 [FIN, ACK] Seq=507 Ack=156 Win=65280 Len=0
834	21.759869	127.0.0.1	127.0.0.1	TCP	44	8000 → 57955 [ACK] Seq=156 Ack=508 Win=65824 Len=0

Figura 12: Captura de tráfico HTTP en Wireshark utilizando el filtro `tcp.port == 8000`.

En esta captura aparece una petición:

```
GET /weather/Granada HTTP/1.1
```

Esta línea confirma que el cliente está consultando el recurso REST encargado de devolver el tiempo de la ciudad de Granada. El servidor responde posteriormente con:

```
HTTP/1.1 200 OK, JSON (application/json)
```

El código 200 OK indica que la operación se ha realizado correctamente. Además, Wireshark identifica que el contenido de la respuesta es de tipo `application/json`, lo que confirma que la API devuelve datos en formato JSON.

También se observan varias peticiones:

```
GET /favicon.ico HTTP/1.1
```

Estas peticiones no forman parte de la lógica principal de MeteoApp. El navegador las genera automáticamente al intentar cargar el icono de la página. Como la API no define un recurso específico para `/favicon.ico`, el servidor responde:

```
HTTP/1.1 404 Not Found
```

Este comportamiento es normal y no representa un fallo en la API. Simplemente indica que el recurso solicitado por el navegador no existe.

11.3. Análisis de la segunda captura

En la Figura 13 se observa tráfico relacionado con el acceso a Swagger y con una petición HTTP no permitida.

1384	42.475312	127.0.0.1	127.0.0.1	HTTP/1.1	75	HTTP/1.1 405 Method Not Allowed, JSON (application/json)
1385	42.475344	127.0.0.1	127.0.0.1	TCP	44	58055 → 8000 [ACK] Seq=477 Ack=186 Win=65280 Len=0
1394	42.585751	127.0.0.1	127.0.0.1	TCP	56	58060 → 8000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1397	42.585837	127.0.0.1	127.0.0.1	TCP	56	8000 → 58060 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1398	42.585850	127.0.0.1	127.0.0.1	TCP	44	58060 → 8000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1400	42.586065	127.0.0.1	127.0.0.1	HTTP	551	GET /favicon.ico HTTP/1.1
1402	42.586090	127.0.0.1	127.0.0.1	TCP	44	8000 → 58060 [ACK] Seq=1 Ack=508 Win=65024 Len=0
1403	42.586094	127.0.0.1	127.0.0.1	TCP	176	8000 → 58060 [PSH, ACK] Seq=1 Ack=508 Win=65024 Len=132 [TCP PDU reassembled in 1405]
1404	42.586071	127.0.0.1	127.0.0.1	TCP	44	58060 → 8000 [ACK] Seq=508 Ack=133 Win=65280 Len=0
1405	42.586045	127.0.0.1	127.0.0.1	HTTP/1.1	66	HTTP/1.1 404 Not Found, JSON (application/json)
1406	42.586085	127.0.0.1	127.0.0.1	TCP	44	58060 → 8000 [ACK] Seq=508 Ack=155 Win=65280 Len=0
1407	47.499584	127.0.0.1	127.0.0.1	TCP	56	8000 → 58055 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1408	47.499510	127.0.0.1	127.0.0.1	TCP	44	58055 → 8000 [ACK] Seq=477 Ack=187 Win=65280 Len=0
1409	47.499588	127.0.0.1	127.0.0.1	TCP	44	58055 → 8000 [FIN, ACK] Seq=477 Ack=187 Win=65280 Len=0
1500	47.499600	127.0.0.1	127.0.0.1	TCP	44	8000 → 58055 [ACK] Seq=187 Ack=478 Win=65024 Len=0
1501	47.499713	127.0.0.1	127.0.0.1	TCP	44	8000 → 58060 [FIN, ACK] Seq=155 Ack=508 Win=65024 Len=0
1502	47.499724	127.0.0.1	127.0.0.1	TCP	44	58060 → 8000 [ACK] Seq=508 Ack=156 Win=65280 Len=0
1503	47.499758	127.0.0.1	127.0.0.1	TCP	44	58060 → 8000 [FIN, ACK] Seq=508 Ack=156 Win=65280 Len=0
1504	47.499756	127.0.0.1	127.0.0.1	TCP	44	8000 → 58060 [ACK] Seq=156 Ack=509 Win=65024 Len=0
1507	47.565295	127.0.0.1	127.0.0.1	TCP	56	63157 → 8000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1508	47.565377	127.0.0.1	127.0.0.1	TCP	56	8000 → 63157 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1509	47.565442	127.0.0.1	127.0.0.1	TCP	44	63157 → 8000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1514	47.568655	127.0.0.1	127.0.0.1	HTTP	513	GET / HTTP/1.1
1515	47.568698	127.0.0.1	127.0.0.1	TCP	44	8000 → 63157 [ACK] Seq=1 Ack=478 Win=65024 Len=0
1516	47.569583	127.0.0.1	127.0.0.1	TCP	176	8000 → 63157 [PSH, ACK] Seq=1 Ack=478 Win=65024 Len=132 [TCP PDU reassembled in 1518]
1517	47.569625	127.0.0.1	127.0.0.1	TCP	44	63157 → 8000 [ACK] Seq=478 Ack=133 Win=65280 Len=0
1518	47.569689	127.0.0.1	127.0.0.1	HTTP/1.1	66	HTTP/1.1 404 Not Found, JSON (application/json)
1519	47.569706	127.0.0.1	127.0.0.1	TCP	44	63157 → 8000 [ACK] Seq=478 Ack=155 Win=65280 Len=0
1526	47.604518	127.0.0.1	127.0.0.1	TCP	56	8000 → 63158 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1527	47.604506	127.0.0.1	127.0.0.1	TCP	56	8000 → 63158 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1528	47.604550	127.0.0.1	127.0.0.1	TCP	44	63158 → 8000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1531	47.604727	127.0.0.1	127.0.0.1	HTTP	544	GET /favicon.ico HTTP/1.1
1532	47.604753	127.0.0.1	127.0.0.1	TCP	44	8000 → 63158 [ACK] Seq=1 Ack=501 Win=65024 Len=0
1533	47.605785	127.0.0.1	127.0.0.1	TCP	176	8000 → 63158 [PSH, ACK] Seq=1 Ack=501 Win=65024 Len=132 [TCP PDU reassembled in 1535]
1534	47.605825	127.0.0.1	127.0.0.1	TCP	44	63158 → 8000 [ACK] Seq=501 Ack=133 Win=65280 Len=0
1535	47.605871	127.0.0.1	127.0.0.1	HTTP/1.1	66	HTTP/1.1 404 Not Found, JSON (application/json)
1536	47.605890	127.0.0.1	127.0.0.1	TCP	44	63158 → 8000 [ACK] Seq=501 Ack=155 Win=65280 Len=0
1721	52.578801	127.0.0.1	127.0.0.1	TCP	44	8000 → 63157 [FIN, ACK] Seq=155 Ack=478 Win=65024 Len=0
1722	52.578856	127.0.0.1	127.0.0.1	TCP	44	63157 → 8000 [ACK] Seq=478 Ack=156 Win=65280 Len=0
1723	52.578278	127.0.0.1	127.0.0.1	TCP	44	63157 → 8000 [FIN, ACK] Seq=478 Ack=156 Win=65280 Len=0
1724	52.578232	127.0.0.1	127.0.0.1	TCP	44	8000 → 63157 [ACK] Seq=156 Ack=471 Win=65024 Len=0
1725	52.609471	127.0.0.1	127.0.0.1	TCP	44	8000 → 63158 [FIN, ACK] Seq=155 Ack=501 Win=65024 Len=0
1726	52.609498	127.0.0.1	127.0.0.1	TCP	44	63158 → 8000 [ACK] Seq=501 Ack=156 Win=65280 Len=0
1727	52.609578	127.0.0.1	127.0.0.1	TCP	44	63158 → 8000 [FIN, ACK] Seq=501 Ack=156 Win=65280 Len=0
1728	52.609586	127.0.0.1	127.0.0.1	TCP	44	8000 → 63158 [ACK] Seq=156 Ack=502 Win=65024 Len=0
3049	99.872085	127.0.0.1	127.0.0.1	TCP	56	8000 → 63371 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3050	99.872082	127.0.0.1	127.0.0.1	TCP	56	8000 → 63371 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3051	99.872046	127.0.0.1	127.0.0.1	TCP	44	63371 → 8000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
3056	99.873935	127.0.0.1	127.0.0.1	HTTP	517	GET /docs HTTP/1.1
3057	99.873117	127.0.0.1	127.0.0.1	TCP	44	8000 → 63371 [ACK] Seq=1 Ack=474 Win=65024 Len=0
3060	99.880418	127.0.0.1	127.0.0.1	TCP	179	8000 → 63371 [PSH, ACK] Seq=1 Ack=474 Win=65024 Len=135 [TCP PDU reassembled in 3062]
3061	99.880456	127.0.0.1	127.0.0.1	TCP	44	63371 → 8000 [ACK] Seq=474 Ack=136 Win=65280 Len=0

Figura 13: Captura de acceso a documentación Swagger y respuesta HTTP de método no permitido.

En la parte inferior de la captura aparece:

```
GET /docs HTTP/1.1
```

Esta petición corresponde al acceso a la documentación automática de FastAPI. Swagger permite visualizar y probar los endpoints de la API desde el navegador, por lo que resulta útil para comprobar el funcionamiento del servidor sin necesidad de escribir un cliente específico.

En la misma captura también aparece una respuesta:

```
HTTP/1.1 405 Method Not Allowed
```

El código 405 Method Not Allowed significa que el cliente ha enviado una petición con un método HTTP no permitido para ese recurso. Por ejemplo, si un endpoint está definido únicamente para POST y se intenta acceder a él mediante GET, el servidor rechaza la operación con este código.

Este comportamiento demuestra que la API no acepta cualquier operación sobre cualquier URL, sino que valida el método HTTP utilizado. Esto es importante en REST, ya que el método forma parte de la semántica de la operación.

De nuevo se observan peticiones a:

```
GET /favicon.ico HTTP/1.1
```

y respuestas:

```
HTTP/1.1 404 Not Found
```

Como se ha comentado anteriormente, estas peticiones son generadas automáticamente por el navegador y no afectan al funcionamiento de la API.

11.4. Análisis de la tercera captura

La Figura 14 muestra la parte más representativa de las pruebas, ya que aparecen las operaciones de creación de cliente, creación de suscripción, gestión de errores y consulta de suscripciones.

3329	189.613180	127.0.0.1	127.0.0.1	TCP	56	63418 → 8000	[SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3330	189.613182	127.0.0.1	127.0.0.1	TCP	56	8000 → 63418	[ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3331	189.613192	127.0.0.1	127.0.0.1	TCP	44	63418 → 8000	[ACK] Seq=1 Ack=1 Win=65280 Len=0
3332	189.613523	127.0.0.1	127.0.0.1	HTTP	508	POST /clients HTTP/1.1	
3333	189.613540	127.0.0.1	127.0.0.1	TCP	44	8000 → 63418	[ACK] Seq=1 Ack=465 Win=65024 Len=0
3336	189.617182	127.0.0.1	127.0.0.1	TCP	174	8000 → 63418	[PSH, ACK] Seq=1 Ack=465 Win=65024 Len=130 [TCP PDU reassembled in 3336]
3337	189.617229	127.0.0.1	127.0.0.1	TCP	44	63418 → 8000	[ACK] Seq=465 Ack=131 Win=65280 Len=0
3338	189.617271	127.0.0.1	127.0.0.1	HTTP/1.1	137	HTTP/1.1 201 Created	, JSON (application/json)
3339	189.617288	127.0.0.1	127.0.0.1	TCP	44	63418 → 8000	[ACK] Seq=465 Ack=224 Win=65280 Len=0
3448	114.627910	127.0.0.1	127.0.0.1	TCP	44	8000 → 63418	[FIN, ACK] Seq=224 Ack=465 Win=65024 Len=0
3441	114.627941	127.0.0.1	127.0.0.1	TCP	44	63418 → 8000	[ACK] Seq=465 Ack=225 Win=65280 Len=0
3442	114.628190	127.0.0.1	127.0.0.1	TCP	44	63418 → 8000	[FIN, ACK] Seq=465 Ack=225 Win=65280 Len=0
3443	114.628154	127.0.0.1	127.0.0.1	TCP	44	8000 → 63418	[ACK] Seq=225 Ack=466 Win=65024 Len=0
4637	175.858393	127.0.0.1	127.0.0.1	TCP	56	63672 → 8000	[SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
4638	175.858470	127.0.0.1	127.0.0.1	TCP	56	8000 → 63672	[SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
4639	175.859515	127.0.0.1	127.0.0.1	TCP	44	63672 → 8000	[ACK] Seq=1 Ack=1 Win=65280 Len=0
4640	175.858733	127.0.0.1	127.0.0.1	HTTP/1.1	663	POST /clients/0dccc75-f3d-4db-a31c-90a2c771cb7d/subscriptions HTTP/1.1	, JSON (application/json)
4641	175.858757	127.0.0.1	127.0.0.1	TCP	44	8000 → 63672	[ACK] Seq=1 Ack=620 Win=64768 Len=0
4664	175.766745	127.0.0.1	127.0.0.1	TCP	175	8000 → 63672	[PSH, ACK] Seq=1 Ack=620 Win=64768 Len=131 [TCP PDU reassembled in 4666]
4665	175.766795	127.0.0.1	127.0.0.1	TCP	44	63672 → 8000	[ACK] Seq=620 Ack=132 Win=65280 Len=0
4666	175.766844	127.0.0.1	127.0.0.1	HTTP/1.1	255	HTTP/1.1 201 Created	, JSON (application/json)
4667	175.766861	127.0.0.1	127.0.0.1	TCP	44	63672 → 8000	[ACK] Seq=620 Ack=343 Win=65024 Len=0
4752	188.781867	127.0.0.1	127.0.0.1	TCP	56	8000 → 63672	[FIN, ACK] Seq=343 Ack=620 Win=64768 Len=0
4753	188.781898	127.0.0.1	127.0.0.1	TCP	44	63672 → 8000	[ACK] Seq=620 Ack=344 Win=65024 Len=0
4754	188.782054	127.0.0.1	127.0.0.1	TCP	44	63672 → 8000	[FIN, ACK] Seq=620 Ack=344 Win=65024 Len=0
4755	188.782101	127.0.0.1	127.0.0.1	TCP	44	8000 → 63672	[ACK] Seq=344 Ack=621 Win=64768 Len=0
5267	200.778090	127.0.0.1	127.0.0.1	TCP	56	63779 → 8000	[SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
5268	200.778172	127.0.0.1	127.0.0.1	TCP	56	8000 → 63779	[SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
5269	200.778222	127.0.0.1	127.0.0.1	TCP	44	63779 → 8000	[ACK] Seq=1 Ack=1 Win=65280 Len=0
5270	200.778381	127.0.0.1	127.0.0.1	HTTP/1.1	663	POST /clients/0dccc75-f3d-4db-a31c-90a2c771cb7d/subscriptions HTTP/1.1	, JSON (application/json)
5271	200.778398	127.0.0.1	127.0.0.1	TCP	44	8000 → 63779	[ACK] Seq=1 Ack=620 Win=64768 Len=0
5274	200.771741	127.0.0.1	127.0.0.1	TCP	178	8000 → 63779	[PSH, ACK] Seq=1 Ack=620 Win=64768 Len=134 [TCP PDU reassembled in 5276]
5275	200.771757	127.0.0.1	127.0.0.1	TCP	44	63779 → 8000	[ACK] Seq=620 Ack=133 Win=65280 Len=0
5276	200.771794	127.0.0.1	127.0.0.1	HTTP/1.1	115	HTTP/1.1 400 Bad Request	, JSON (application/json)
5277	200.771801	127.0.0.1	127.0.0.1	TCP	44	63779 → 8000	[ACK] Seq=620 Ack=206 Win=65280 Len=0
5452	205.772246	127.0.0.1	127.0.0.1	TCP	44	8000 → 63779	[FIN, ACK] Seq=206 Ack=620 Win=65024 Len=0
5453	205.772789	127.0.0.1	127.0.0.1	TCP	44	63779 → 8000	[ACK] Seq=620 Ack=207 Win=65280 Len=0
5454	205.772847	127.0.0.1	127.0.0.1	TCP	44	63779 → 8000	[FIN, ACK] Seq=620 Ack=207 Win=65280 Len=0
5455	205.773023	127.0.0.1	127.0.0.1	TCP	44	8000 → 63779	[ACK] Seq=207 Ack=621 Win=64768 Len=0
6283	248.793924	127.0.0.1	127.0.0.1	TCP	56	64620 → 8000	[SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
6284	248.793989	127.0.0.1	127.0.0.1	TCP	56	8000 → 64620	[SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
6285	248.794019	127.0.0.1	127.0.0.1	TCP	44	64620 → 8000	[ACK] Seq=1 Ack=1 Win=65280 Len=0
6286	248.794183	127.0.0.1	127.0.0.1	HTTP	508	GET /clients/0dccc75-f3d-4db-a31c-90a2c771cb7d/subscriptions HTTP/1.1	
6287	248.794199	127.0.0.1	127.0.0.1	TCP	44	8000 → 64620	[ACK] Seq=1 Ack=465 Win=65024 Len=0
6290	248.795566	127.0.0.1	127.0.0.1	TCP	170	8000 → 64620	[PSH, ACK] Seq=1 Ack=465 Win=65024 Len=126 [TCP PDU reassembled in 6292]
6291	248.795594	127.0.0.1	127.0.0.1	TCP	44	64620 → 8000	[ACK] Seq=465 Ack=127 Win=65280 Len=0
6292	248.795620	127.0.0.1	127.0.0.1	HTTP/1.1	241	HTTP/1.1 200 OK	, JSON (application/json)
6293	248.795629	127.0.0.1	127.0.0.1	TCP	44	64620 → 8000	[ACK] Seq=465 Ack=324 Win=65024 Len=0
6380	253.766851	127.0.0.1	127.0.0.1	TCP	44	8000 → 64620	[FIN, ACK] Seq=324 Ack=465 Win=65024 Len=0
6381	253.766889	127.0.0.1	127.0.0.1	TCP	44	64620 → 8000	[ACK] Seq=465 Ack=325 Win=65024 Len=0
6382	253.767052	127.0.0.1	127.0.0.1	TCP	44	64620 → 8000	[FIN, ACK] Seq=465 Ack=325 Win=65024 Len=0
6383	253.767106	127.0.0.1	127.0.0.1	TCP	44	8000 → 64620	[ACK] Seq=325 Ack=466 Win=65024 Len=0

Figura 14: Captura de peticiones POST y GET sobre clientes y suscripciones.

En primer lugar, se observa la petición:

```
POST /clients HTTP/1.1
```

El servidor responde con:

```
HTTP/1.1 201 Created
```

Esto confirma que la creación del cliente REST se ha realizado correctamente. El código 201 Created es el adecuado porque se ha creado un nuevo recurso en el servidor.

Posteriormente aparece una petición de creación de suscripción:

```
POST /clients/0dccc75-f3d-4db-a31c-90a2c771cb7d/subscriptions HTTP/1.1
```

La respuesta asociada es:

```
HTTP/1.1 201 Created
```

Esto indica que el cliente identificado por `client_id` se ha suscrito correctamente a una ciudad. Además, Wireshark muestra que el contenido de la petición es JSON, lo que confirma que el cuerpo de la solicitud viaja en formato legible.

Más adelante se observa una segunda petición POST al mismo recurso de suscripciones, seguida de una respuesta:

```
HTTP/1.1 400 Bad Request
```

Este error es coherente si se ha intentado crear una suscripción duplicada. La API detecta que el cliente ya estaba suscrito a esa ciudad y rechaza la petición. Por tanto, la captura no solo muestra casos correctos, sino también la gestión de errores de la API.

Finalmente, se observa una petición:

```
GET /clients/0dccc75-f3d-4db-a31c-90a2c771cb7d/subscriptions HTTP/1.1
```

El servidor responde con:

```
HTTP/1.1 200 OK
```

Esto confirma que la consulta de las suscripciones del cliente funciona correctamente. En este caso no se crea ni modifica ningún recurso, sino que se obtiene información ya existente, por lo que el método GET y el código 200 OK son adecuados.

11.5. Interpretación global de las capturas

Las capturas permiten verificar varios aspectos importantes de la práctica:

- El servidor FastAPI está escuchando en el puerto 8000.
- La comunicación se realiza mediante HTTP.
- Las operaciones utilizan métodos HTTP adecuados: GET para consultar y POST para crear.
- Las respuestas incluyen códigos de estado coherentes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found y 405 Method Not Allowed.
- Los datos se transmiten en formato JSON, indicado mediante `application/json`.
- El tráfico es legible y puede interpretarse directamente desde Wireshark.

Esta última diferencia es especialmente relevante respecto a RMI. En RMI, el tráfico aparece asociado a JRMI y a serialización Java, por lo que resulta menos transparente para el análisis manual. En cambio, en la versión REST se puede observar claramente qué recurso se ha solicitado, qué método se ha usado y qué código de respuesta ha devuelto el servidor.

12. Problemas encontrados y soluciones adoptadas

Durante la transformación del sistema se han identificado varios problemas técnicos.

El primer problema ha sido traducir métodos remotos a recursos REST sin caer en un diseño RPC disfrazado. Para resolverlo, se han modelado las entidades principales del sistema como recursos: ciudades, tiempo meteorológico, clientes, suscripciones y notificaciones.

El segundo problema ha sido adaptar el callback de RMI. En la versión anterior, el servidor podía notificar directamente al cliente. En REST, este comportamiento se ha sustituido por una cola de notificaciones consultable mediante `GET /clients/{client_id}/notifications`. Esta solución respeta el modelo petición-respuesta de HTTP.

El tercer problema ha sido la identificación de los clientes. Como HTTP no mantiene estado de sesión por defecto, se ha creado un recurso cliente mediante `POST /clients`. El identificador generado permite asociar suscripciones y notificaciones a cada cliente.

También se ha abordado la gestión de errores. Por ejemplo, si falta la clave de OpenWeather, el servidor no puede realizar la consulta meteorológica externa y devuelve un error. Si se intenta acceder a un recurso inexistente, se devuelve `404 Not Found`. Si se repite una suscripción ya existente, se devuelve `400 Bad Request`. Y si se utiliza un método HTTP incorrecto, se devuelve `405 Method Not Allowed`.

13. Conclusiones

La práctica ha permitido transformar el sistema MeteoApp desde un modelo basado en sockets y objetos distribuidos hacia una arquitectura REST basada en HTTP y JSON. La lógica funcional se ha mantenido: se pueden consultar ciudades, obtener información meteorológica, crear clientes, gestionar suscripciones y consultar notificaciones.

La principal diferencia está en la forma de comunicación. RMI oculta gran parte de la comunicación remota y permite trabajar con métodos y callbacks. REST, en cambio, obliga a diseñar recursos, URLs, métodos HTTP y códigos de estado. Este cambio hace que la API sea más interoperable, más fácil de documentar y más sencilla de probar con herramientas estándar.

Las capturas de Wireshark confirman que la nueva versión utiliza tráfico HTTP sobre el puerto 8000, con mensajes JSON y códigos de estado adecuados. Esto demuestra que la transformación se ha realizado correctamente y que la comunicación puede analizarse de forma clara, a diferencia de lo que ocurre con la serialización propia de RMI.

En definitiva, la versión REST mantiene la semántica del sistema original, pero la adapta a un modelo más abierto, documentado y compatible con clientes heterogéneos.